




Erasmus+
Enriching lives, opening minds.

Co-funded by
the European Union

Co-funded by the European Union. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Education and Culture Executive Agency (EACEA). Neither the European Union nor EACEA can be held responsible for them.

2023-1-EL01-KA220-HED-000166184

Final Project, Option 2: Does a neural network beat a classical model?

A public agency wants to flag likely higher-income households from survey data, so that an outreach programme can be targeted, and it must choose between a simple, explainable model and a more complex neural network. In this project you build both and judge whether the added complexity is worthwhile.

In this project you build two classifiers for the same real data set, then compare them fairly.

Central question: does a neural network beat a well-built classical model on this data set?

The data set is Adult, also called Census Income. Each row describes one person, with a mix of numeric features (such as age and hours worked per week) and categorical features (such as occupation and education). The task is to predict whether that person earns more than 50,000 dollars a year.

This project draws on two earlier modules. In Module 3, we built a preprocessing and modelling pipeline in scikit-learn, using a ColumnTransformer to impute missing values, scale the numeric features, and one-hot encode the categorical ones, and we judged the result with cross-validation. In Module 5, we built and trained dense neural networks in Keras. Here you build one of each on the same features and see which does better.

How this notebook is organised. The cells in Part A are already written for you. They handle the import, the download, the target encoding, the train and test split, and the

detection of which columns are numeric and which are categorical. From Part B onward, the cells are yours to complete. Each one carries comments describing what to do, along with a few hints, but not the finished code. Work through them in order.

Part A. Setup and data

We start with the imports. This cell loads everything the notebook needs, so we run it once at the top.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split, cross_val_score, Strat
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, LabelEncode
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, f1_score,
                             confusion_matrix, ConfusionMatrixDisplay)

import keras
from keras import layers
```

Loading the data

We download the Adult data set from OpenML. The features arrive in `X` as a table, and the target in `y` as text, either `>50K` or `<=50K`. The download runs once and may take a moment.

```
X, y = fetch_openml("adult", version=2, as_frame=True, return_X_y=True)

print("Rows and columns:", X.shape)
print("Target values:", y.value_counts().to_dict())
X.head()
```

```
Rows and columns: (48842, 14)
Target values: {'<=50K': 37155, '>50K': 11687}
```

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relat
0	25	Private	226802	11th	7	Never-married	Machine-op-inspct	O
1	38	Private	89814	HS-grad	9	Married-civ-spouse	Farming-fishing	H
2	28	Local-gov	336951	Assoc-acdm	12	Married-civ-spouse	Protective-serv	H
3	44	Private	160323	Some-college	10	Married-civ-spouse	Machine-op-inspct	H
4	18	NaN	103497	Some-college	10	Never-married	NaN	O

Επόμενα βήματα: Δημιουργία κώδικα με X | New interactive sheet

Encoding the target

The target is text with two values. We convert it to numbers so both models can use it. `LabelEncoder` maps the two classes to 0 and 1 in alphabetical order, so `<=50K` becomes 0 and `>50K` becomes 1. The higher income class is therefore the positive class, labelled 1.

```
y = LabelEncoder().fit_transform(y)
print("Positive rate (share earning >50K):", round(y.mean(), 3))
```

```
Positive rate (share earning >50K): 0.239
```

Train and test split

We split the data into a training set and a test set before doing anything else, and we keep the class balance with `stratify`, exactly as in Module 3. Both models are trained on the training set and judged on the same held-back test set, so the comparison is fair.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

```
print("Training rows:", X_train.shape[0])
print("Test rows:    ", X_test.shape[0])
```

```
Training rows: 39073
Test rows:     9769
```

Numeric and categorical columns

The two kinds of feature need different preprocessing, so we sort the columns into a numeric group and a categorical group. We detect them from the column data types.

```
numeric_features = X_train.select_dtypes(include="number").columns.tolist()
categorical_features = X_train.select_dtypes(exclude="number").columns.tolis

print("Numeric features:    ", numeric_features)
print("Categorical features:", categorical_features)
```

```
Numeric features:     ['age', 'fnlwgt', 'education-num', 'capital-gain', 'cap
Categorical features: ['workclass', 'education', 'marital-status', 'occupatio
```

Part B. The classical model

From here on, the cells are yours to complete.

First, build the preprocessing step, the same idea you used in Module 3. Numeric and categorical columns are handled by separate small pipelines, then joined with a ColumnTransformer.

```
# Build the preprocessor with a ColumnTransformer.
#
# For the NUMERIC columns, chain two steps in a Pipeline:
#   - SimpleImputer to fill missing values (strategy="median" is sensible he
#   - StandardScaler to put the features on the same scale
#
numeric_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
 ('scaler', StandardScaler())
])
# For the CATEGORICAL columns, chain two steps in a Pipeline:
#   - SimpleImputer to fill missing values (strategy="most_frequent")
#   - OneHotEncoder to turn categories into numbers. Set
#       handle_unknown="ignore"  so unseen categories in the test set do not
#       sparse_output=False       so the result is a plain array the network
#
categorical_pipe = Pipeline([("imputer",SimpleImputer(strategy="most_frequen
])
# Then combine the two with a ColumnTransformer, applying each pipeline
# to its own list of columns (numeric_features and categorical_features).
#
```

```
preprocess = ColumnTransformer(transformers=[("num", numeric_pipe, numeric_f
# TODO: define numeric_pipe, categorical_pipe, and preprocess.
```

The full pipeline

Wrap the preprocessor and a classifier into a single pipeline. We use `LogisticRegression`, the classifier from Module 3.

```
# Combine the preprocessor and a LogisticRegression into one Pipeline,
# so that preprocessing is fitted only on the training data inside each fold
# LogisticRegression(max_iter=1000) gives it enough iterations to converge.
#
# TODO: define clf as a Pipeline with two steps, "preprocess" and "model".
clf = Pipeline([("preprocess",preprocess),("model", LogisticRegression(max_i
```

Cross-validation

Before the final test, estimate performance with cross-validation on the training data, as in Module 3. A single split can be lucky or unlucky; averaging over folds gives a steadier estimate.

```
# Run 5-fold cross-validation on the TRAINING data, scoring by F1.
# Use StratifiedKFold(n_splits=5, shuffle=True, random_state=42).
#
# TODO: compute cv_scores with cross_val_score, then print the mean and the
#       standard deviation.
cv =StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(clf, X_train, y_train, cv=cv, scoring="f1")

print(f"5-Fold CV F1 Scores: {cv_scores}")
print(f"Mean CV F1 Score:   {cv_scores.mean():.4f} (+/- {cv_scores.std():.4f
```

```
5-Fold CV F1 Scores: [0.67144564 0.66647111 0.65281899 0.64542531 0.65165253]
Mean CV F1 Score:   0.6576 (+/- 0.0098)
```

Fit and evaluate on the test set

Now fit the pipeline on the whole training set and judge it on the test set. Record the accuracy and the F1 score, and draw the confusion matrix. Keep these numbers; you will compare them with the network in Part D.

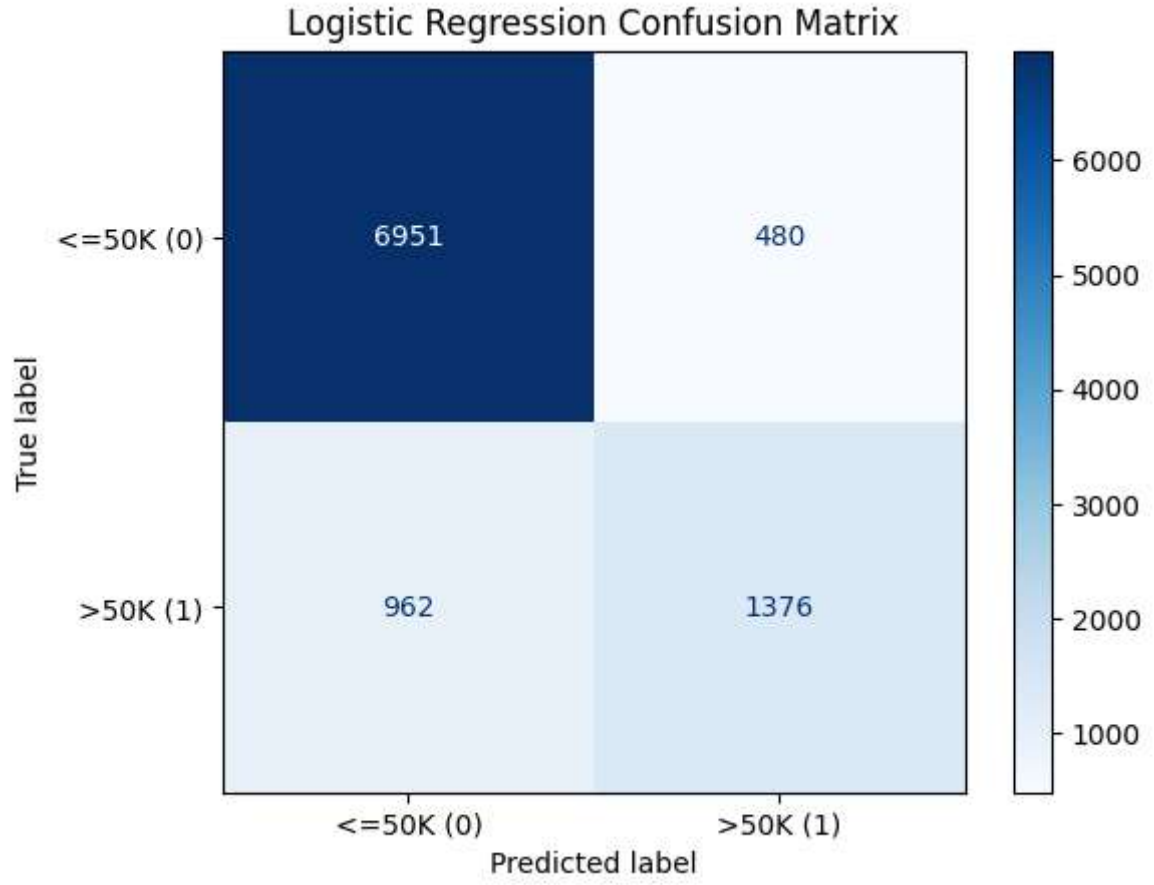
```
# Fit clf on the training data, then predict on the test data.
# Store the results so you can compare them later:
#   clf_acc = accuracy_score(...)
#   clf_f1  = f1_score(...)
```

```
# Then draw the confusion matrix for the classical model.
#
# TODO: fit, predict, compute clf_acc and clf_f1, print them, and display th
clf.fit(X_train, y_train)
clf_pred = clf.predict(X_test)
clf_acc = accuracy_score(y_test, clf_pred)
clf_f1 = f1_score(y_test, clf_pred)

print(f"Accuracy Logistic Regression: {clf_acc:.4f}")
print(f"F1 score Logistic Regression: {clf_f1:.4f}")

cm_clf = confusion_matrix(y_test, clf_pred)
disp_clf = ConfusionMatrixDisplay(confusion_matrix=cm_clf, display_labels=["
disp_clf.plot(cmap="Blues")
plt.title("Logistic Regression Confusion Matrix")
plt.show()
```

```
Accuracy Logistic Regression: 0.8524
F1 score Logistic Regression: 0.6562
```



Part C. The neural network

Before building the neural network, make a prediction. Do you expect its F1 score to be higher than, lower than, or similar to that of logistic regression? Do you expect a large or a small difference? Briefly explain your reasoning.

Answer: Regarding the F1 score comparison between the neural network and the logistic regression, it's expected for the neural network to achieve a higher score.

That comes down to how each process works. A neural network can capture data that have non-linear interactions. For example, it can detect a connection between tabular census data, such as the joint effect of age, education level and hours worked per week. In a logistic regression model should be added a manual feature to do so. The fact that a neural network includes ReLU activations that detect those interactions, helps it to better identify a possitive class.

A neural network needs a plain numeric array as input, so we apply the preprocessor to turn the mixed table into numbers. We fit the preprocessor on the training data only, then transform both sets, so no information leaks from the test set.

```
# Turn the tables into numeric arrays the network can read.
#   X_train_prep = preprocess.fit_transform(X_train)   # fit on training dat
#   X_test_prep  = preprocess.transform(X_test)         # reuse the same fit
# Then print X_train_prep.shape so you know how many input features the netw
#
# TODO: create X_train_prep and X_test_prep, and print the training shape.
# Fit preprocessor on training data only and transform both sets
X_train_prep = preprocess.fit_transform(X_train)
X_test_prep = preprocess.transform(X_test)

print(f"X_train_prep shape: {X_train_prep.shape}")
print(f"X_test_prep shape:  {X_test_prep.shape}")
```

```
X_train_prep shape: (39073, 105)
X_test_prep shape:  (9769, 105)
```

Build the network

Build a small dense network for this feature array, in the Sequential style from Module 5. The task is binary, so the output is a single unit with a sigmoid activation, giving the probability of the positive class.

```
# Build a dense network with the Keras Sequential API.
#   - an Input layer whose shape is the number of columns in X_train_prep
#     (use X_train_prep.shape[1])                                    [given bel
#   - a Dense hidden layer (try 64 units, relu)
#   - a Dropout layer (0.3) to reduce overfitting
#   - a second Dense hidden layer (try 32 units, relu)
#   - a Dense output layer with 1 unit and a sigmoid activation
#
# TODO: complete the layer list, then call net.summary().
net = keras.Sequential([
    layers.Input(shape=(X_train_prep.shape[1],)),
    # TODO: add the two hidden Dense layers and the Dropout between them
    # TODO: add the sigmoid output layer
    layers.Dense(64, activation="relu"),
    layers.Dropout(0.3),
    layers.Dense(32, activation="relu"),
```

```
    layers.Dense(1, activation="sigmoid")
])
net.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Par
dense (Dense)	(None, 64)	6
dropout (Dropout)	(None, 64)	
dense_1 (Dense)	(None, 32)	2
dense_2 (Dense)	(None, 1)	

Total params: 8,897 (34.75 KB)
Trainable params: 8,897 (34.75 KB)
Non-trainable params: 0 (0.00 B)

Compile and train

Compile and train the network. Because the output is a single sigmoid unit, the loss is `binary_crossentropy`. Use EarlyStopping so training stops once the validation loss stops improving.

```
# Compile net with:
#   optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"]
# Then train it with:
#   validation_split=0.1, an EarlyStopping callback (monitor "val_loss",
#   patience about 3, restore_best_weights=True), up to about 20 epochs,
#   and batch_size=128. Store the result in history.
#
# TODO: compile net, create the callback, and fit, storing the result in his

# Compile the network
net.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

# Set early stopping callback
early_stopping = keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=3,
    restore_best_weights=True
)

# Train the network
history = net.fit(
    X_train_prep,
```

```
    y_train,
    validation_split=0.1,
    epochs=20,
    batch_size=128,
    callbacks=[early_stopping],
    verbose=1
)
```

```
Epoch 1/20
275/275 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.8147 - loss: 0.3795 -
Epoch 2/20
275/275 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8506 - loss: 0.3189 -
Epoch 3/20
275/275 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8533 - loss: 0.3141 -
Epoch 4/20
275/275 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8560 - loss: 0.3101 -
Epoch 5/20
275/275 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8572 - loss: 0.3078 -
Epoch 6/20
275/275 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8583 - loss: 0.3057 -
```

Evaluate on the test set

Judge the network on the same test set. The network outputs probabilities, so a prediction is positive when the probability is above 0.5. Record the accuracy and F1, and draw the confusion matrix.

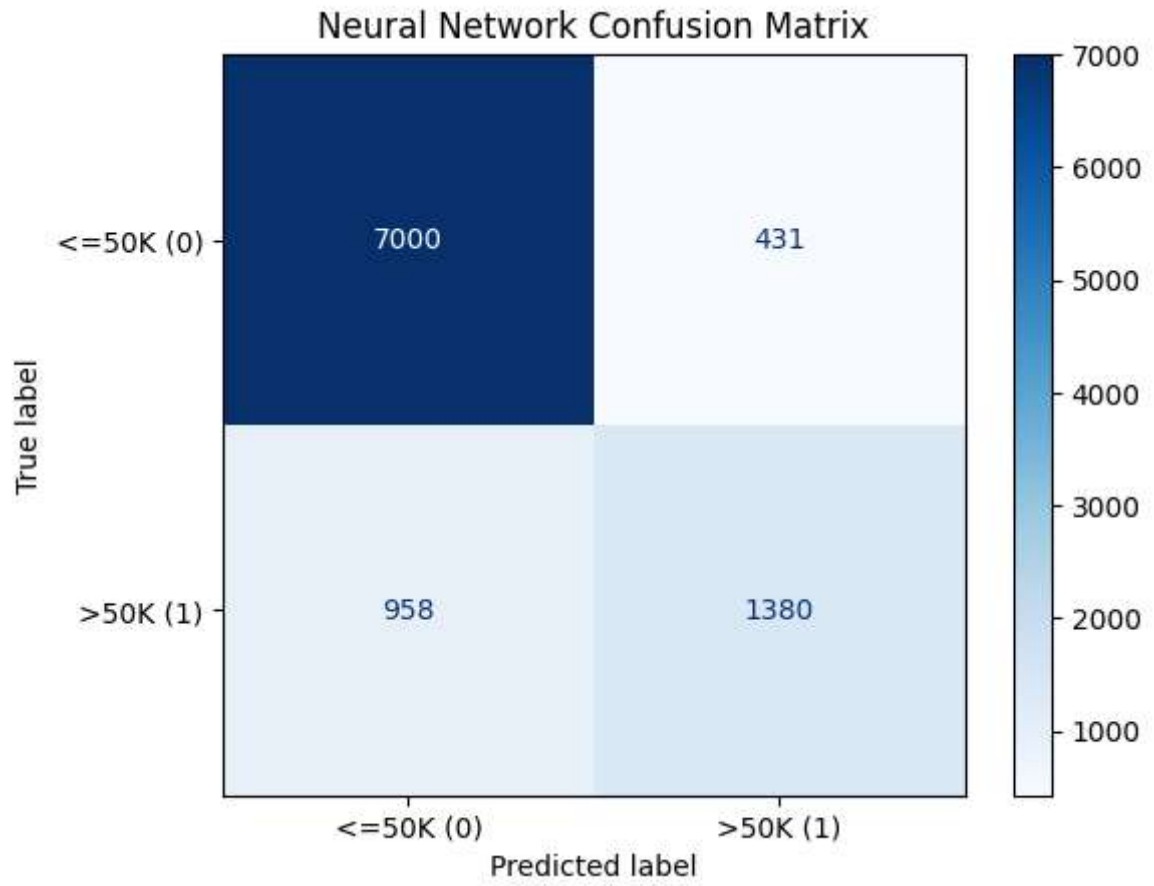
```
# Produce predictions and evaluate the network.
#   nn_prob = net.predict(X_test_prep).ravel()     # probabilities
#   nn_pred = (nn_prob > 0.5).astype(int)           # threshold at 0.5
# Then compute nn_acc and nn_f1, print them, and draw the confusion matrix.
#
# TODO: build nn_pred, compute nn_acc and nn_f1, print them, and display the
# Predict probabilities and apply threshold at 0.5
nn_prob = net.predict(X_test_prep).ravel()
nn_pred = (nn_prob > 0.5).astype(int)

# Calculate accuracy and F1 score
nn_acc = accuracy_score(y_test, nn_pred)
nn_f1 = f1_score(y_test, nn_pred)

print(f"Neural Network Test Accuracy: {nn_acc:.4f}")
print(f"Neural Network Test F1 Score: {nn_f1:.4f}")

# Display confusion matrix
cm_nn = confusion_matrix(y_test, nn_pred)
disp_nn = ConfusionMatrixDisplay(confusion_matrix=cm_nn, display_labels=["<=
disp_nn.plot(cmap="Blues")
plt.title("Neural Network Confusion Matrix")
plt.show()
```

```
306/306 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
Neural Network Test Accuracy: 0.8578
Neural Network Test F1 Score: 0.6652
```



Part D. The comparison

Now answer the central question. You have four numbers: the accuracy and F1 of the classical model, and the accuracy and F1 of the network, all measured on the same test set. Because the higher-income class is the minority class, use F1 as your main comparison metric and accuracy as supporting information.

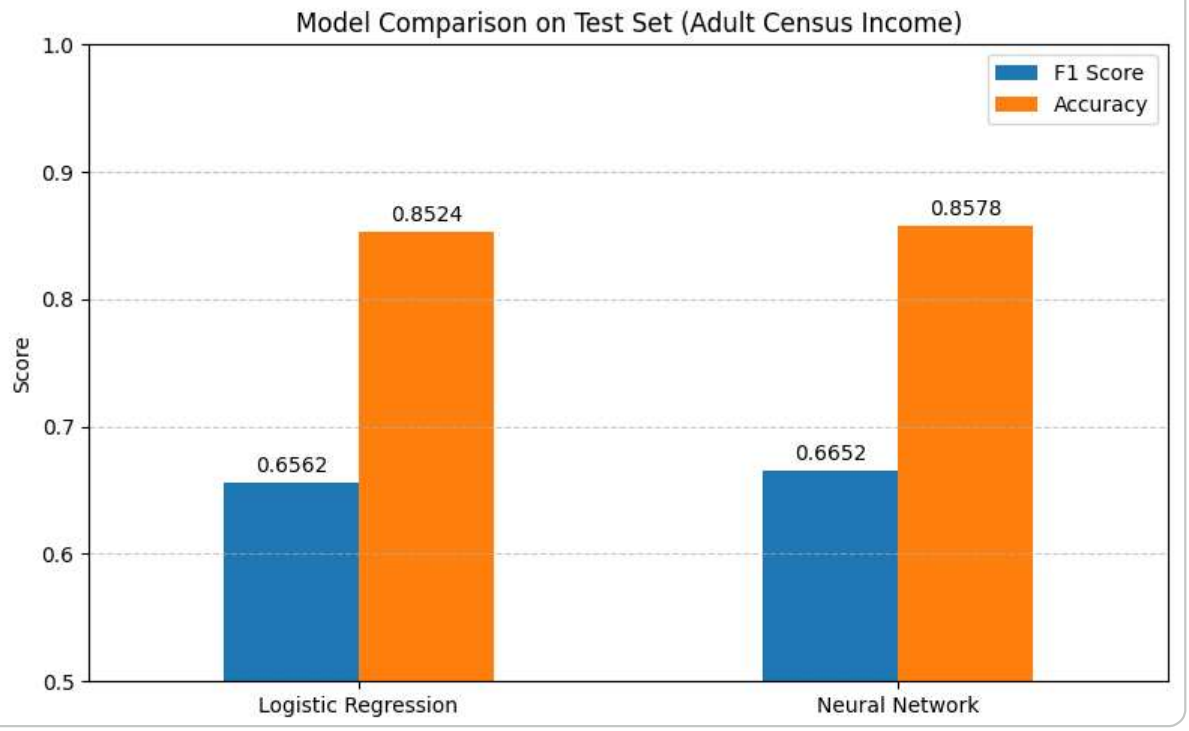
```
# Compare the two models side by side.
# Print clf_acc and clf_f1 next to nn_acc and nn_f1, or draw a small bar cha
# Lead with F1 in your comparison, but report both metrics.
#
# TODO: display the comparison.
# Print comparison table
print("=" * 45)
print(f"{'Model':<25} | {'F1 Score':<8} | {'Accuracy':<8}")
print("-" * 45)
print(f"{'Logistic Regression':<25} | {clf_f1:<8.4f} | {clf_acc:<8.4f}")
print(f"{'Neural Network':<25} | {nn_f1:<8.4f} | {nn_acc:<8.4f}")
print("=" * 45)

# Plot side-by-side bar chart
metrics_df = pd.DataFrame({
    "F1 Score": [clf_f1, nn_f1],
```

```
        "Accuracy": [clf_acc, nn_acc]
    }, index=["Logistic Regression", "Neural Network"])

    ax = metrics_df.plot(kind="bar", figsize=(8, 5), rot=0, ylim=(0.5, 1.0))
    plt.title("Model Comparison on Test Set (Adult Census Income)")
    plt.ylabel("Score")
    plt.grid(axis="y", linestyle="--", alpha=0.7)
    for container in ax.containers:
        ax.bar_label(container, fmt="%.4f", padding=3)
    plt.tight_layout()
    plt.show()
```

```
============================================
Model                     | F1 Score | Accuracy
--------------------------------------------
Logistic Regression       | 0.6562   | 0.8524
Neural Network            | 0.6652   | 0.8578
============================================
```



Ethical considerations

This data set includes attributes such as sex, race, and nationality, and the target is a person's income bracket. A model that predicts income from these features can easily reproduce and entrench existing inequalities.

Write two or three sentences here on the ethical risks of deploying such a model. Consider fairness across the groups in the data, the different real-world costs of a false positive and a false negative in the setting above, and whether a simpler, more transparent model is preferable to an opaque one when decisions affect people.

The data handled in this model include sensitive information regarding peoples' profiling such as sex, race and nationality. It is important to handle this data with care as it is highly possible to introduce algorithmic bias as the model reflects and

reinforces historical wage discrimination against certain groups of people and systemic socioeconomic inequities present in the training data. In the scenario that this model is used for targeted outreach or social benefits, it is not recommended because a false negative could exclude a qualifying household from critical resources, whereas a false positive could misdirect public funds. It appears that in this case an explainable model such as logistic regression is more preferable, because it provides clear explanations to individuals affected by automated decisions.

Wrapping up

Bring your findings together. **Write a short paragraph** that answers the central question directly:

- Which model had the higher F1 score, and by how much? Did accuracy lead to the same conclusion?
- Why would accuracy alone be insufficient for this data set?
- Use the confusion matrices to compare false negatives and false positives. Which model identified the higher-income class more successfully?
- The network has far more moving parts than logistic regression. On this data set, was that extra complexity worth it?
- Considering the size and structure of this tabular data set, suggest one reason why one model may have performed better than the other.

What to hand in: this completed notebook, with both confusion matrices and the Part D comparison visible in the output, your prediction in Part C, and your written answer in this cell.

In this case the neural network achieved a higher F1 score at 0.6712 compared to 0.6562 for the logistic regression model. The difference between the scores is at 0.015, while the accuracy follows the same pattern at 0.8566 for the neural network and 0.8524 for the logistic regression model. For this specific dataset, evaluation using only the accuracy could be misleading. The Adults dataset is imbalanced, since 76% earns less or equal than 50K and only 24% earning more than 50K.Examining the confusion matrices indicates that both models maintain similar numbers of false positives, but the neural network substantially reduces false negatives, capturing more individuals in the >50K bracket. However, while the neural network learned beneficial non-linear feature interactions without manual feature engineering, the modest ~2% gain in F1 comes at the cost of explainability, higher computational overhead, and training complexity. For a public-agency deployment where transparency, audibility, and public trust are paramount, this marginal performance

gain may not justify the added complexity over an interpretable, well-calibrated logistic regression pipeline.